SQL Practice Notes

Authored by: Arpit Raj, Lnmiit jaipur

Stadium

```
Table: Stadium
+---------------+---------+
| Column Name   | Type    |
+---------------+---------+
| id            | int     |
| visit_date    | date    |
| people        | int     |
+---------------+---------+
visit_date is the column with unique values for this table.
Each row of this table contains the visit date and visit id to the stadium with the n
As the id increases, the date increases as well.
```

Write a solution to display the records with three or more rows with consecutive id's, and the number of people is greater than or equal to 100 for each.
Return the result table ordered by visit_date in ascending order.
The result format is in the following example.

Example 1:

Input:

```
Stadium table:
+------+------------+-----------+
| id   | visit_date | people    |
+------+------------+-----------+
| 1    | 2017-01-01 | 10        |
| 2    | 2017-01-02 | 109       |
| 3    | 2017-01-03 | 150       |
| 4    | 2017-01-04 | 99        |
| 5    | 2017-01-05 | 145       |
| 6    | 2017-01-06 | 1455      |
| 7    | 2017-01-07 | 199       |
| 8    | 2017-01-09 | 188       |
+------+------------+-----------+
```

Output:

```
+------+------------+-----------+
| id   | visit_date | people    |
+------+------------+-----------+
| 5    | 2017-01-05 | 145       |
| 6    | 2017-01-06 | 1455      |
| 7    | 2017-01-07 | 199       |
| 8    | 2017-01-09 | 188       |
+------+------------+-----------+
```

Explanation:
The four rows with ids 5, 6, 7, and 8 have consecutive ids and each of them has >= 100 people attended. Note that row 8 was included even though the visit_date was not the next day after row 7.
The rows with ids 2 and 3 are not included because we need at least three consecutive ids.

Query:

```
SELECT DISTINCT s1.*FROM Stadium s1JOIN Stadium s2ON s2.id = s1.id + 1JOIN Stadium s3ON s3.id = s1.id + 2WHERE s1.people >= 100AND s2.people >= 100AND s3.people >= 100UNIONSELECT DISTINCT s2.*FROM Stadium s1JOIN Stadium s2ON s2.id = s1.id + 1JOIN Stadium s3ON s3.id = s1.id + 2WHERE s1.people >= 100AND s2.people >= 100AND s3.people >= 100UNIONSELECT DISTINCT s3.*FROM Stadium s1JOIN Stadium s2ON s2.id = s1.id + 1JOIN Stadium s3ON s3.id = s1.id + 2WHERE s1.people >= 100AND s2.people >= 100AND s3.people >= 100ORDER BY id;
```

The UNION is needed because each query returns only one position of the valid 3-day sequence.
Let's see why.
Suppose the table is
idpeople512061307140
These are 3 consecutive days with people >= 100.

First query
`SELECT s1.*`
returns
id5
because
s1 = 5
s2 = 6
s3 = 7
So only s1 is selected.

Second query
`SELECT s2.*`
returns
id6

Third query
`SELECT s3.*`
returns
id7

Now combine them
First Query
5
UNION
Second Query
6
UNION
Third Query
7
↓
id567
Exactly what the problem wants.

Activity

```
Table: Activity
+--------------+---------+
| Column Name  | Type    |
+--------------+---------+
| player_id    | int     |
| device_id    | int     |
| event_date   | date    |
| games_played | int     |
+--------------+---------+
(player_id, event_date) is the primary key (combination of columns with unique values
This table shows the activity of players of some games.
Each row is a record of a player who logged in and played a number of games (possibly
```

Write a solution to find the first login date for each player.
Return the result table in any order.
The result format is in the following example.

Example 1:

Input:

```
Activity table:
+-----------+-----------+------------+--------------+
| player_id | device_id | event_date | games_played |
+-----------+-----------+------------+--------------+
| 1         | 2         | 2016-03-01 | 5            |
| 1         | 2         | 2016-05-02 | 6            |
| 2         | 3         | 2017-06-25 | 1            |
| 3         | 1         | 2016-03-02 | 0            |
| 3         | 4         | 2018-07-03 | 5            |
+-----------+-----------+------------+--------------+
```

Output:

```
+-----------+-------------+
| player_id | first_login |
+-----------+-------------+
| 1         | 2016-03-01  |
| 2         | 2017-06-25  |
| 3         | 2016-03-02  |
+-----------+-------------+
```

Query:

```mysql
# Write your MySQL query statement below
SELECT
    player_id,
    MIN(event_date) AS first_login
FROM Activity
GROUP BY player_id;
```

Orders

```
Table: Orders
+-----------------+----------+
| Column Name     | Type     |
+-----------------+----------+
| order_number    | int      |
| customer_number | int      |
+-----------------+----------+
order_number is the primary key (column with unique values) for this table.
This table contains information about the order ID and the customer ID.
```

Write a solution to find the customer_number for the customer who has placed the largest number of orders.
The test cases are generated so that exactly one customer will have placed more orders than any other customer.
The result format is in the following example.

Example 1:

Input:

```
Orders table:
+--------------+-----------------+
| order_number | customer_number |
+--------------+-----------------+
| 1            | 1               |
| 2            | 2               |
| 3            | 3               |
| 4            | 3               |
+--------------+-----------------+
```

Output:

```
+-----------------+
| customer_number |
+-----------------+
| 3               |
+-----------------+
```

Explanation:
The customer with number 3 has two orders, which is greater than either customer 1 or 2 because each of them only has one order.
So the result is customer_number 3.

ActorDirector

```
able: ActorDirector
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| actor_id    | int     |
| director_id | int     |
| timestamp   | int     |
+-------------+---------+
timestamp is the primary key (column with unique values) for this table.
```

Write a solution to find all the pairs (actor_id, director_id) where the actor has cooperated with the director at least three times.

Return the result table in any order.
The result format is in the following example.

Example 1:

Input:

```
ActorDirector table:
+-------------+-------------+-------------+
| actor_id    | director_id | timestamp   |
+-------------+-------------+-------------+
| 1           | 1           | 0           |
| 1           | 1           | 1           |
| 1           | 1           | 2           |
| 1           | 2           | 3           |
| 1           | 2           | 4           |
| 2           | 1           | 5           |
| 2           | 1           | 6           |
+-------------+-------------+-------------+
```

Output:

```
+-------------+-------------+
| actor_id    | director_id |
+-------------+-------------+
| 1           | 1           |
+-------------+-------------+
```

> **Explanation:** The only pair is (1, 1) where they cooperated exactly 3 times.

Query:

```
SELECT
    actor_id,
    director_idFROM ActorDirectorGROUP BY actor_id, director_idHAVING
COUNT(*) >= 3;
```

Queries

```
Table: Queries
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| query_name  | varchar |
| result      | varchar |
| position    | int     |
| rating      | int     |
+-------------+---------+
This table may have duplicate rows.
This table contains information collected from some queries on a database.
The position column has a value from 1 to 500.
The rating column has a value from 1 to 5. Query with rating less than 3 is a poor qu
```

We define query quality as:
The average of the ratio between query rating and its position.

We also define poor query percentage as:
The percentage of all queries with rating less than 3.

Write a solution to find each query_name, the quality and poor_query_percentage.
Both quality and poor_query_percentage should be rounded to 2 decimal places.
Return the result table in any order.
The result format is in the following example.

Example 1:

Input:

```
Queries table:
+------------+-------------------+----------+--------+
| query_name | result            | position | rating |
+------------+-------------------+----------+--------+
| Dog        | Golden Retriever  | 1        | 5      |
| Dog        | German Shepherd   | 2        | 5      |
| Dog        | Mule              | 200      | 1      |
| Cat        | Shirazi           | 5        | 2      |
| Cat        | Siamese           | 3        | 3      |
| Cat        | Sphynx            | 7        | 4      |
+------------+-------------------+----------+--------+
```

Output:

```
+------------+---------+-----------------------+
| query_name | quality | poor_query_percentage |
+------------+---------+-----------------------+
| Dog        | 2.50    | 33.33                 |
| Cat        | 0.66    | 33.33                 |
+------------+---------+-----------------------+
```

Explanation:
Dog queries quality is ((5 / 1) + (5 / 2) + (1 / 200)) / 3 = 2.50
Dog queries poor_ query_percentage is (1 / 3) * 100 = 33.33

Cat queries quality equals ((2 / 5) + (3 / 3) + (4 / 7)) / 3 = 0.66
Cat queries poor_ query_percentage is (1 / 3) * 100 = 33.33 or each query_name, return:

quality
Average(ratingposition) ext{Average}\left(rac{ ext{rating}}{ ext{position}} ight)
Average(position
rating
)
rounded to 2 decimal places.

Query:

```
# Write your MySQL query statement below
SELECT
    query_name,

    ROUND(AVG(rating / position),2) AS quality,

    ROUND(
        SUM(CASE WHEN rating<3 THEN 1 ELSE 0 END)
        *100
        /COUNT(*),
        2
    ) AS poor_query_percentage

FROM Queries

GROUP BY query_name;
```

Project

```
Table: Project
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| project_id  | int     |
| employee_id | int     |
+-------------+---------+
(project_id, employee_id) is the primary key of this table.
employee_id is a foreign key to Employee table.
Each row of this table indicates that the employee with employee_id is working on th

Table: Employee
+------------------+---------+
| Column Name      | Type    |
+------------------+---------+
| employee_id      | int     |
| name             | varchar |
| experience_years | int     |
+------------------+---------+
employee_id is the primary key of this table. It's guaranteed that experience_years
Each row of this table contains information about one employee.
```

Write an SQL query that reports the average experience years of all the employees for each project, rounded to 2 digits.
Return the result table in any order.
The query result format is in the following example.

Example 1:

Input:

```
Project table:
+-------------+-------------+
| project_id  | employee_id |
+-------------+-------------+
| 1           | 1           |
| 1           | 2           |
| 1           | 3           |
| 2           | 1           |
| 2           | 4           |
+-------------+-------------+
Employee table:
+-------------+--------+------------------+
| employee_id | name   | experience_years |
+-------------+--------+------------------+
| 1           | Khaled | 3                |
| 2           | Ali    | 2                |
| 3           | John   | 1                |
| 4           | Doe    | 2                |
+-------------+--------+------------------+
```

Output:

```
+-------------+---------------+
| project_id  | average_years |
+-------------+---------------+
| 1           | 2.00          |
| 2           | 2.50          |
+-------------+---------------+
```

Explanation: The average experience years for the first project is (3 + 2 + 1) / 3 = 2.00 and for the second project is (3 + 2) / 2 = 2.50

Query:

```
SELECT
    p.project_id,
    ROUND(AVG(e.experience_years),2) AS average_yearsFROM Project
pJOIN Employee eON p.employee_id = e.employee_idGROUP BY p.project_id;
```

MyNumbers

```
Table: MyNumbers
+-------------+------+
| Column Name | Type |
+-------------+------+
| num         | int  |
+-------------+------+
This table may contain duplicates (In other words, there is no primary key for this t
Each row of this table contains an integer.
```

A single number is a number that appeared only once in the MyNumbers table.
Find the largest single number. If there is no single number, report null.
The result format is in the following example.

Example 1:

Input:

```
MyNumbers table:
+-----+
| num |
+-----+
| 8   |
| 8   |
| 3   |
| 3   |
| 1   |
| 4   |
| 5   |
| 6   |
+-----+
```

Output:

```
+-----+
| num |
+-----+
| 6   |
+-----+
```

> **Explanation:** The single numbers are 1, 4, 5, and 6.
> Since 6 is the largest single number, we return it.

Example 2:

Input:

```
MyNumbers table:
+-----+
| num |
+-----+
| 8   |
| 8   |
| 7   |
| 7   |
| 3   |
| 3   |
| 3   |
+-----+
```

Output:

```
+------+
| num  |
+------+
| null |
+------+
```

> **Explanation:** There are no single numbers in the input table so we return null.

Query:

```
SELECT numFROM MyNumbersGROUP BY numHAVING COUNT(*) = 1ORDER BY num
DESCLIMIT 1;
```

> Why isn't this enough?
> If there is no single number, this query returns 0 rows.
> But the problem expects
>
> NULL

Triangle

```
Table: Triangle
+-------------+------+
| Column Name | Type |
+-------------+------+
| x           | int  |
| y           | int  |
| z           | int  |
+-------------+------+
In SQL, (x, y, z) is the primary key column for this table.
Each row of this table contains the lengths of three line segments.
```

Report for every three line segments whether they can form a triangle.
Return the result table in any order.
The result format is in the following example.

Example 1:

Input:

```
Triangle table:
+----+----+----+
| x  | y  | z  |
+----+----+----+
| 13 | 15 | 30 |
| 10 | 20 | 15 |
+----+----+----+
```

Output:

```
+----+----+----+----------+
| x  | y  | z  | triangle |
+----+----+----+----------+
| 13 | 15 | 30 | No       |
| 10 | 20 | 15 | Yes      |
+----+----+----+----------+
```

Query:

```
SELECT
    x,
    y,
    z,
    CASE
        WHEN x+y>z
         AND x+z>y
         AND y+z>x
        THEN 'Yes'
        ELSE 'No'
    END AS triangleFROM Triangle;
```

Seat

```
Table: Seat
+-------------+---------+
| Column Name | Type    |
+-------------+---------+
| id          | int     |
| student     | varchar |
+-------------+---------+
id is the primary key (unique value) column for this table.
Each row of this table indicates the name and the ID of a student.
The ID sequence always starts from 1 and increments continuously.
```

Write a solution to swap the seat id of every two consecutive students. If the number of students is odd, the id of the last student is not swapped.
Return the result table ordered by id in ascending order.
The result format is in the following example.

Example 1:

Input:

```
Seat table:
+----+---------+
| id | student |
+----+---------+
| 1  | Abbot   |
| 2  | Doris   |
| 3  | Emerson |
| 4  | Green   |
| 5  | Jeames  |
+----+---------+
```

Output:

```
+----+---------+
| id | student |
+----+---------+
| 1  | Doris   |
| 2  | Abbot   |
| 3  | Green   |
| 4  | Emerson |
| 5  | Jeames  |
+----+---------+
```

Explanation:
Note that if the number of students is odd, there is no need to change the last one's seat.

Query:

```
# Write your MySQL query statement below
SELECT
    CASE
        WHEN id % 2 = 1 AND id != (SELECT MAX(id) FROM Seat)
            THEN id + 1
        WHEN id % 2 = 0
            THEN id - 1
        ELSE id
    END AS id,
    student
FROM Seat
ORDER BY id;
```